Hamster's Stash

倉鼠小金庫

Erin's Technical Task Guide

34 Tasks · 5 Phases · ~59% of Total Work

Flutter · Riverpod · Isar · TDD · CI/CD

架構總覽 (Architecture Overview)

Tech Stack

- **Framework:** Flutter 3.2x+
- **State:** Riverpod + riverpod_annotation (code gen)
- **Database:** Isar Database (AES-256 加密，key 存 Keychain/Keystore)
- **Network:** Dio + dio_smart_retry (Exponential Backoff, 3 retries, 10s timeout)
- **Navigation:** go_router (宣告式路由)
- **Charts:** fl_chart (圓餅圖、長條圖、折線圖)
- **Testing:** mockito + mocktail + integration_test + very_good_analysis
- **CI:** GitHub Actions (lint → test → coverage ≥ 80% → build → deploy)

Feature-First 目錄結構

```
lib/
  core/                # 共用工具、常數、擴展方法
    constants/          # API URLs, 固定參數
    errors/             # 自定義 Exception
    extensions/         # Dart extension methods
    utils/              # 格式化、轉換工具
  features/
    accounts/           # 帳戶 CRUD、assetTerm 分類
      data/             # Repository impl, Data sources, DTOs
      domain/           # Entities, Repository interface, Use cases
      presentation/     # Screens, Widgets, Riverpod providers
    transactions/       # 交易記錄 CRUD、轉帳
    categories/         # 大小分類 CRUD 與選擇器
    budget/             # 預算設定、進度追蹤、超支提醒
    recurring/          # 週期性交易規則與排程
    reports/            # 圓餅圖、長條圖、折線圖
    calendar/           # 月曆檢視
    search/             # 搜尋、篩選、排序
```

```
    data_export/        # CSV 匯出
    asset_overview/     # 資產總覽、淨資產計算
    market_data/        # 股價 API
    invoice_scanner/    # 發票掃描
    exchange_rate/      # 匯率 API、快取、手動覆蓋
    receivables_payables/   # 應收應付
    auth_lock/          # 生物辨識鎖
  app.dart              # MaterialApp 設定
  main.dart             # Entry point
test/
  unit/    widget/    integration/    e2e/    mocks/    fixtures/
```

資料模型總覽 (Data Schema)

Account, Transaction, Category, Budget, RecurringRule, Receivable/Payable, ManualValuation 的完整 schema 請見開發規格書。以下列出核心 schema：

Account

欄位	型別	說明
`id`	Id	主鍵
`name`	String	如: Firstrade, 國泰世華
`type`	Enum	Cash, Bank, Investment, CreditCard, RealEstate, Vehicle, Insurance, Other
`currency`	String	TWD, USD
`assetTerm`	Enum	current (短期) / nonCurrent (長期)
`balance`	Double	快取餘額，避免每次重算
`stockSymbols`	List<String>	持有股票代碼
`icon`	String	自定義圖示 (emoji)
`isArchived`	Boolean	軟刪除

Transaction

欄位	型別	說明
`id`	Id	主鍵
`type`	Enum	expense / income / transfer
`amount`	Double	原始交易金額
`currency`	String	交易幣別
`exchangeRate`	Double	最終使用的匯率
`isManualRate`	Boolean	是否手動輸入匯率
`categoryId`	Int	大分類 ID
`subcategoryId`	Int?	小分類 ID (可選)
`accountId`	Id	所屬帳戶

欄位	型別	說明
toAccountId	Id?	轉帳目標帳戶 (僅 transfer)
dateTime	DateTime	交易時間 (建 Isar Index)
note	String?	備註
tags	List<String>	自定義標籤
photoPath	String?	收據照片本地路徑
stockSymbol	String?	股票代碼
invoiceNumber	String?	發票號碼
recurringId	Id?	關聯週期性交易 ID

Receivable / Payable

欄位	型別	說明
id	Id	主鍵
type	Enum	receivable (應收) / payable (應付)
counterparty	String	對象姓名
totalAmount	Double	總金額
settledAmount	Double	已收/已付金額，預設 0
currency	String	幣別
status	Enum	pending / partial / settled
dueDate	DateTime?	到期日 (可選)
note	String?	備註
linkedTransactionIds	List<Id>	關聯的 Transaction ID (自動沖銷)
createdAt	DateTime	建立時間
updatedAt	DateTime	更新時間

🛠️ **與 Emma 的協作模式**

Phase 1–2：Emma 用 Claude Code 做 UI (假資料)，你建後端邏輯。兩人平行不衝突。
Phase 3：你將 Emma 的 UI 接上真實資料。Emma 提供分類樹 CSV + 帳戶分類對照表 + 淨資產驗算。
Phase 4：Emma 提供「會計規格書」，你照著實作。建議 Emma 的 4-E1~4-E5 先完成。
Phase 5：Emma 做會計 QA，你修她找到的 bug。

🚀 Phase 0：專案初始化 (Day 1)

全部由你完成，建立專案基礎架構，讓 Emma 可以接下來直接用 Claude Code 開始做 UI。

0-1	建立 **Flutter** 專案	⭐⭐⭐⭐

Objective

建立專案骨架並安裝所有主要依賴。

Steps

- **flutter create hamsters_stash** --org com.yourname
- 安裝主要依賴：

```
# pubspec.yaml dependencies:
flutter_riverpod, riverpod_annotation, riverpod_generator
isar, isar_flutter_libs, path_provider
go_router, fl_chart, dio, dio_smart_retry
local_auth, camera, google_ml_kit_barcode_scanning
flutter_secure_storage, flutter_local_notifications
table_calendar, csv, share_plus, home_widget
intl, logger

# dev_dependencies:
riverpod_lint, build_runner, isar_generator
mockito, mocktail, very_good_analysis, integration_test
```

- 建立 **Feature-First** 目錄結構 (見架構總覽)
- 設定 **go_router** 基本路由：

```
final router = GoRouter(routes: [
  GoRoute(path: '/', builder: (_, __) => SplashScreen()),
  StatefulShellRoute.indexedStack(
    branches: [
      // /overview, /transaction, /reports, /settings
    ],
  ),
]);
```

Acceptance Criteria

- flutter run 成功啟動，顯示空白 App
- 所有依賴安裝成功，無編譯錯誤
- go_router 4 個 Tab 切換正常

0-2	**Isar** 資料庫設定	⭐⭐⭐⭐

Objective

定義所有 Isar Collection schema，初始化加密資料庫。

Steps

- 定義以下 **Collection**：

```
@collection
class Account { Id id; String name; AccountType type; String currency;
  AssetTerm assetTerm; double balance; List<String> stockSymbols;
  String icon; bool isArchived; }

@collection
class Transaction { Id id; TransactionType type; double amount;
  String currency; double exchangeRate; bool isManualRate;
  int categoryId; int? subcategoryId; Id accountId; Id? toAccountId;
  DateTime dateTime; String? note; List<String> tags;
  String? photoPath; String? stockSymbol; String? invoiceNumber;
  Id? recurringId; DateTime createdAt; DateTime updatedAt; }

// 同樣定義: Category, Budget, RecurringRule,
// ReceivablePayable, ManualValuation
```

- 建立 **Isar Index**：

```
// Transaction 必建索引：
@Index([IndexField('dateTime')])
@Index([IndexField('categoryId')])
@Index([IndexField('accountId')])
```

- 初始化加密：

```
final encryptionKey = await _getOrCreateKey(); // flutter_secure_storage
final isar = await Isar.open(
  [AccountSchema, TransactionSchema, CategorySchema,
   BudgetSchema, RecurringRuleSchema, ReceivablePayableSchema,
   ManualValuationSchema],
  encryptionKey: encryptionKey,
);
```

Acceptance Criteria

- build_runner 產生所有 .g.dart 檔案無錯誤
- Isar 開啟成功，基本 CRUD 可運作
- encryption key 存儲於 Keychain/Keystore

0-3	主題設計系統 (Theme)	⭐⭐⭐⭐

Objective

建立「可愛保險箱」風格的設計 Token。

Design Tokens

```
// colors
primary: #D4740A (warm orange)
primaryLight: #FFF3E6
secondary: #2E5090 (deep blue)
expense: #E74C3C (red)
income: #2ECC71 (green)
surface: #FFFFFF, background: #FFF8F0 (warm off-white)

// typography
displayLarge: 28pt bold  (page titles)
titleMedium: 18pt semi   (card titles)
```

```
bodyMedium: 14pt regular (body text)
labelSmall: 12pt         (captions)

// shape
borderRadius: 16 (cards), 24 (buttons), 999 (chips)
elevation: 2 (cards), 4 (FAB)

// 支援 Dark Mode (Phase 5-4 實作)
// 先用 ThemeData.light() 建立，留好擴展點
```

Acceptance Criteria

- ThemeData 全局可用，所有頁面風格一致
- 支援 MediaQuery.textScaleFactor 縮放

0-4	**Git Repo 設定**	⭐⭐⭐

Steps

- 建立 GitHub repo (private)
- Branch rules: main (protected) → develop → feature/*
- .gitignore 加入 Isar .isar 檔、*.g.dart (or not, 依團隊決定)
- 請 Emma clone 並安裝 Claude Code
- 確認 Emma 可以 flutter run 成功

0-5	預設分類 **Seed Data**	⭐⭐⭐

Objective

建立預設的收入/支出分類樹，首次安裝時自動載入。Phase 3 會用 Emma 提供的完整版替換。

Implementation

```
Future<void> seedDefaultCategories(Isar isar) async {
  if (await isar.categorys.count() > 0) return; // 已有資料則跳過
  final defaults = [
    Category(name: '飲食', icon: '🍔', color: '#FF6B6B',
            type: CategoryType.expense, sortOrder: 0),
    // ... 其他預設分類
  ];
  await isar.writeTxn(() => isar.categorys.putAll(defaults));
}
```

> ⏰ **Phase 3 依賴**
> Emma 的 3-E1 會提供完整的分類樹 CSV (含 emoji、色碼)，屆時用它替換這裡的簡易版。

🔧 Phase 2：核心後端邏輯 (Day 5–10)

Emma 同時在做 UI 元件 (用假資料)，你建立後端資料層。兩人平行不衝突。

> 🎯 **每個 Feature 的標準實作模式**
> 1. Domain Layer: Entity + Repository interface + Use cases
> 2. Data Layer: IsarDataSource + RepositoryImpl
> 3. Presentation Layer: Riverpod providers (StateNotifier / AsyncNotifier)
> 4. Testing: 先寫 unit test (Given-When-Then)，再實作

2-A	**Category CRUD**	⭐⭐⭐

Implementation

- 建立 CategoryRepository (abstract) + IsarCategoryRepository (impl)
- **Riverpod provider:**

```
@riverpod
class CategoryNotifier extends _$CategoryNotifier {
  @override
  Future<List<Category>> build() => _repo.getAll();
  Future<void> add(Category c) => ...
  Future<void> update(Category c) => ...
  Future<void> delete(Id id) => ...
  List<Category> getChildren(Id parentId) => ...
}
```

- 父子關係邏輯：刪除大分類時聯動刪除所有小分類
- 預設 **seed** 資料載入：首次啟動時自動塞入預設分類

Testing

```
test('given parent category deleted, when checking children, then all children removed')
test('given seed data loaded, when querying, then default categories exist')
test('given expense category, when querying income, then not included')
```

2-B	**Account CRUD + assetTerm 分類**	⭐⭐⭐⭐

Implementation

- AccountRepository + Riverpod AsyncNotifier
- **assetTerm 分類邏輯**：

```
// 預設規則 (後續用 Emma 的 3-E2 替換)
AssetTerm defaultTerm(AccountType type) => switch(type) {
  Cash || Bank || Investment => AssetTerm.current,
  RealEstate || Vehicle || Insurance => AssetTerm.nonCurrent,
  CreditCard => AssetTerm.liability,  // 特殊處理
  _ => AssetTerm.current,
};
```

- 餘額計算：balance 欄位快取，每次 Transaction CRUD 時聯動更新
- 軟刪除：isArchived = true，保留歷史交易參照

Testing

```
test('given new Investment account, when created, then assetTerm is current')
test('given account with transactions, when archived, then balance preserved')
test('given 3 accounts, when querying by assetTerm, then correctly grouped')
```

2-C	Transaction CRUD	⭐⭐⭐⭐

Implementation

- 核心邏輯：新增/編輯/刪除 **Transaction** 時必須聯動更新 **Account.balance**

```
Future<void> addTransaction(Transaction tx) async {
  await isar.writeTxn(() async {
    await isar.transactions.put(tx);
    // 更新來源帳戶餘額
    final account = await isar.accounts.get(tx.accountId);
    if (tx.type == TransactionType.expense) {
      account!.balance -= tx.amount;
    } else if (tx.type == TransactionType.income) {
      account!.balance += tx.amount;
    } else if (tx.type == TransactionType.transfer) {
      account!.balance -= tx.amount;
      final toAccount = await isar.accounts.get(tx.toAccountId!);
      toAccount!.balance += tx.amount * tx.exchangeRate;
      await isar.accounts.put(toAccount);
    }
    await isar.accounts.put(account!);
  });
}
```

- 編輯時需先 **revert** 舊金額，再 **apply** 新金額
- 刪除時需 **revert** 金額影響

Testing

```
test('given expense added, when checking account, then balance decreased')
test('given transfer TWD->USD, when checking both accounts, then balances correct')
test('given transaction deleted, when checking account, then balance reverted')
test('given transaction edited from 100 to 200, then balance diff is 100')
```

2-D	Unit Tests (Batch 1)	⭐⭐⭐⭐

Scope

為 Category / Account / Transaction 的 Use case 和 Repository 寫完整單元測試。

Mock Strategy

```
// 用 mocktail 建立 mock
class MockIsarDatabase extends Mock implements Isar {}
class FakeCategoryRepo extends Fake implements CategoryRepository {}

// Riverpod testing
final container = ProviderContainer(overrides: [
  categoryRepoProvider.overrideWithValue(FakeCategoryRepo()),
```

```
});
```

測試命名慣例

採用 Given-When-Then：

```
test('given USD account, when adding TWD transaction, then auto-suggest exchange rate')
test('given transfer between TWD and USD, then neither counts as income or expense')
```

Coverage Target

目標 ≥ 80%，尤其是餘額計算、轉帳邏輯、分類父子關係必須 100% 覆蓋。

2-E	轉帳邏輯 (Transfer)	⭐⭐⭐

Implementation

- type = TransactionType.transfer
- accountId = 來源帳戶，toAccountId = 目標帳戶
- 跨幣別轉帳：來源扣 amount，目標加 amount * exchangeRate
- 不計入收支統計：報表查詢時過濾 type != transfer

Edge Cases

- 同一帳戶轉給自己：應禁止或警告
- 匯率為 0 或負數：應拒絕
- 同幣別轉帳：exchangeRate 預設 1.0

🔗 Phase 3：前後端串接 (Day 11–15)

將 Emma 做好的 UI 與你建立的資料層連接。

> ⚠️ **依賴 Emma 的交付物**
> 3-E1: 分類樹 CSV (含支出/收入、大小分類、emoji、色碼) → 你的 3-5, 3-6
> 3-E2: 帳戶分類對照表 (哪些 assetTerm = current/nonCurrent) → 你的 3-5
> 3-E3: 淨資產驗算 Excel (測試案例) → 你的單元測試

3-1	總覽頁接上真實資料	⭐⭐⭐

將 Emma 的靈態總覽頁改用 Riverpod provider 讀取真實 Account 資料。

```
// 在 Emma 的 overview_screen.dart 中：
// 將寫死的 $123,456 替換為：
final accounts = ref.watch(accountListProvider);
final totalAssets = accounts.where((a) => !a.isArchived)
    .fold(0.0, (sum, a) => sum + a.balance);

// 短期/長期分類顯示：
final current = accounts.where((a) => a.assetTerm == AssetTerm.current);
final nonCurrent = accounts.where((a) => a.assetTerm == AssetTerm.nonCurrent);
```

3-2	記帳表單接上儲存	⭐⭐⭐

將 Bottom Sheet 的「儲存」按鈕接上 Transaction CRUD，分類選擇器接 Category provider。

- 分類選擇器：GridView 改讀 ref.watch(categoryListProvider)
- 金額輸入：計算機結果傳入 Transaction.amount
- 儲存按鈕：ref.read(transactionNotifierProvider.notifier).add(tx)
- 儲存後：Snackbar 提示 + 關閉 Bottom Sheet + 總覽頁自動更新

3-3	交易列表接上真實資料	⭐⭐⭐

將 Emma 的假資料 ListView 替換為 Riverpod watch Transaction list。

```
final transactions = ref.watch(recentTransactionsProvider(limit: 20));
// AsyncValue handling: loading / data / error
// 分頁: infinite scroll, 每次 30 筆
```

3-4	報表頁接上真實資料	⭐⭐⭐

圓餅圖接上實際分類彙總，計算各分類支出佔比。

```
@riverpod
Future<Map<Category, double>> expenseByCategory(ref, {DateTime? month}) {
  final txns = await _repo.getByMonth(month ?? DateTime.now());
```

```
  return txns.where((t) => t.type == TransactionType.expense)
    .groupBy((t) => t.categoryId)
    .map((catId, txns) => MapEntry(cat, txns.sum((t) => t.amount)));
}
```

3-5	新增帳戶 + 分類管理 UI	⭐⭐⭐

參考 **Emma** 交付物：3-E1 (分類樹) + 3-E2 (帳戶對照表)

- 建立「新增帳戶」表單：名稱、類型、幣別、初始餘額、assetTerm
- 建立「分類管理」頁面：大小分類 CRUD + 拖拉排序 + emoji/顏色選擇
- 帳戶類型預設 **assetTerm**：依據 Emma 的 3-E2 對照表自動帶入

3-6	導入 Emma 的分類 Seed	⭐⭐⭐

將 Emma 提供的 CSV 轉為 Isar seed data。

```
// 讀取 Emma 的 CSV 並轉為 Category list
Future<void> importCategorySeed(String csvPath) async {
  final csv = await rootBundle.loadString(csvPath);
  final rows = const CsvToListConverter().convert(csv);
  // rows: [類型, 大分類, 小分類, emoji, 色碼]
  // 先建大分類, 再建小分類 (參考 parentId)
}
```

🚀 Phase 4：進階功能 (Day 16–25)

最大的開發階段，10 個任務。Emma 同時提供會計規格書。

> ⚠️ **依賴 Emma 的會計規格**
> 4-E1 預算規則 → 你的 4-C
> 4-E2 應收應付流程 → 你的 4-E
> 4-E3 報表指標定義 → 你的 4-G
> 4-E4 轉帳匯率驗算 → 你的 4-J 測試案例
> 4-E5 CSV 格式定義 → 你的 4-I

4-A	匯率 **API** 串接	⭐⭐⭐⭐

Implementation

```
// ExchangeRateRepository
class ExchangeRateService {
  final Dio _dio;
  final Isar _isar; // 快取

  /// 提供建議匯率，快取 30 分鐘
  Future<double> getRate(String from, String to) async {
    final cached = await _getCachedRate(from, to);
    if (cached != null && !cached.isExpired) return cached.rate;
    try {
      // frankfurter.app/latest?from=USD&to=TWD
      final resp = await _dio.get(
        'https://api.frankfurter.app/latest',
        queryParameters: {'from': from, 'to': to},
      );
      final rate = resp.data['rates'][to];
      await _cacheRate(from, to, rate);
      return rate;
    } catch (e) {
      return cached?.rate ?? 1.0; // fallback
    }
  }

  /// 手動覆蓋：isManualRate = true 後停止自動更新
}
```

離線優先

- 快取至 Isar，網路斷線時顯示上次更新時間與快取資料
- Exponential Backoff: 最多 3 次，10s timeout

4-B	股價 **API** 串接	⭐⭐⭐⭐

Implementation

- 代碼自動識別:

```
String resolveSymbol(String input) {
  if (RegExp(r'^\d{4,6}$').hasMatch(input)) {
    return '$input.TW'; // 台股
  }
  return input.toUpperCase(); // 美股
}

// Yahoo Finance API (yfinance or HTTP scraping)
// GET https://query1.finance.yahoo.com/v8/finance/chart/TSLA
// debounce 300ms 避免頻繁呼叫
```

- 現值計算:(持股數 × 即時股價) × 匯率, 顯示在資產總覽

4-C	**Budget 邏輯**	⭐⭐⭐

參考 **Emma** 交付物:4-E1 預算規則設計

Implementation

- Budget CRUD + Riverpod provider
- 消耗計算:查詢當月該分類的 expense 總和 / budget.amount
- 進度條:> 80% 變色警告, > 100% 爭紅
- 超支通知:flutter_local_notifications
- **carryOver** 邏輯:依 Emma 的 4-E1 規則實作過月結轉

4-D	**Recurring 邏輯 (週期性交易)**	⭐⭐⭐⭐

Implementation

```
// RecurringRule schema (see spec Table 5)
// 排程器: 每次 App 啟動時檢查
Future<void> processRecurringRules() async {
  final rules = await _repo.getActiveRules();
  for (final rule in rules) {
    while (rule.nextOccurrence.isBefore(DateTime.now())) {
      // 自動產生 Transaction (from templateTransaction)
      await addTransaction(rule.templateTransaction.copyWith(
        dateTime: rule.nextOccurrence,
        recurringId: rule.id,
      ));
      rule.advanceToNext(); // 更新 nextOccurrence
      // 檢查 endDate / maxOccurrences
      if (rule.isCompleted) { rule.deactivate(); break; }
    }
  }
}
```

- 提前通知:執行前一天發送本地通知提醒

4-E	應收應付 **CRUD**	⭐⭐⭐⭐

參考 **Emma** 交付物：4-E2 應收應付流程審核

狀態機

```
pending —[部分收款]→ partial —[全部收完]→ settled
pending —[一次收完]→ settled

// settledAmount 追蹤累計收款
void recordPayment(double amount) {
  settledAmount += amount;
  if (settledAmount >= totalAmount) status = Status.settled;
  else status = Status.partial;
  // 產生關聯 Transaction (if autoReconcile)
}
```

- 自動沖銷：settled 時產生對應 Transaction，更新 linkedTransactionIds
- 到期提醒：flutter_local_notifications，到期日前提醒
- 總覽整合：淨應收/應付餘額納入淨資產計算

4-F	**ManualValuation CRUD**	⭐⭐⭐

針對無即時報價的資產 (不動產、車輛)，提供「更新現值」功能。

```
// ManualValuation schema (see spec Table 10)
// accountId, valuationAmount, currency, valuationDate, note

// 總覽頁整合：最新估值 = 排序後的第一筆
double getLatestValuation(Id accountId) {
  return valuations.where().accountIdEqualTo(accountId)
    .sortByValuationDateDesc().findFirst()?.valuationAmount ?? 0;
}
```

4-G	報表功能實作	⭐⭐⭐

參考 **Emma** 交付物：4-E3 報表指標設計

三種圖表

- 圓餅圖 **(fl_chart PieChart)**：各分類支出佔比，點擊展開小分類
- 長條圖 **(BarChart)**：每月/每週收支對比
- 折線圖 **(LineChart)**：淨資產走勢、資產變動率

指標公式 (依 Emma 4-E3)

- 儲蓄率 = (總收入 − 總支出) / 總收入 × 100%
- 淨資產 = 總資產 (短期 + 長期) − 總負債 (信用卡 + 應付)
- 資產變動率 = (本月淨資產 − 上月淨資產) / 上月淨資產 × 100%

4-H	搜尋與篩選	⭐⭐⭐

Implementation

- 關鍵字搜尋:note, category name, account name
- 多條件篩選:日期範圍、金額範圍、分類、帳戶、標籤
- 排序:依日期、金額升/降序

```
// Isar query builder pattern
var query = isar.transactions.where();
if (dateRange != null) query = query.dateTimeBetween(start, end);
if (categoryId != null) query = query.categoryIdEqualTo(categoryId);
if (keyword != null) query = query.noteContains(keyword);
return query.sortByDateTimeDesc().findAll();
```

4-I	資料匯出 CSV	⭐⭐⭐

參考 **Emma** 交付物:4-E5 CSV 格式定義

```
// 使用 csv package
final csvData = const ListToCsvConverter().convert([
  ['Date', 'Type', 'Category', 'Subcategory', 'Amount', 'Currency', ...]
  ...transactions.map((t) => [
    t.dateTime.toIso8601String(),
    t.type.name,
    getCategoryName(t.categoryId),
    ...
  ]),
]);

// 透過 share_plus 分享
final file = File('${tempDir.path}/export.csv');
await file.writeAsString(csvData);
await Share.shareXFiles([XFile(file.path)]);
```

- 篩選匯出:支援依時間區間、帳戶、分類篩選後再匯出

4-J	Unit Tests (Batch 2)	⭐⭐⭐⭐

匯率、Budget、Recurring、應收應付、資產分類單元測試。

重要:用 **Emma** 的驗算表作為測試案例!

Key Test Cases

```
// 從 Emma 的 3-E3 驗算表:
test('given Emma scenario 1, then net worth = X')
test('given Emma scenario 2 (multi-currency), then total assets = Y')

// 從 Emma 的 4-E4 匯率驗算:
test('given TWD->USD transfer at rate 0.032, then amounts correct')

// 從 Emma 的 4-E1 預算規則:
test('given carryOver enabled and 500 remaining, then next month budget = X+500')

// 從 Emma 的 4-E2 應收應付流程:
test('given receivable 3000, partial 1000, then settledAmount=1000 status=partial')
```

✨ Phase 5：收尾與拋光 (Day 26–30)

Emma 同時進行會計 QA，你負責進階功能、修 bug、CI/CD。

5-1	發票掃描 (Invoice Scanner)	⭐⭐⭐⭐

```
// google_ml_kit barcode scanning
// 台灣電子發票 QR Code 解析：
// 前 2 碼：字軌，10 碼：發票號碼
// 後綴：Hex 金額 → Decimal
int hexToDecimal(String hex) => int.parse(hex, radix: 16);

// 隱私：全程本地處理，不上傳影像
```

5-2	生物辨識鎖 (Auth Lock)	⭐⭐⭐⭐

```
// AuthService 狀態機：locked / unlocked / unavailable
// App lifecycle 監聽：
WidgetsBindingObserver.didChangeAppLifecycleState(state) {
  if (state == AppLifecycleState.resumed) {
    triggerBiometricAuth();
  }
}

// 背景時清除敏感 UI 資料（避免 task switcher 截圖）
```

5-3	Widget 小工具	⭐⭐⭐⭐

- home_widget package
- **iOS WidgetKit + Android App Widget**：
- 快速記帳 **Widget**：點擊直接開啟記帳頁
- 餘額總覽 **Widget**：顯示當月支出/餘額/預算概況

5-4	暗色模式 (Dark Mode)	⭐⭐⭐

- 新增 ThemeData.dark() 配置
- 確保對比度符合 WCAG AA (≥ 4.5:1)
- 所有頁面測試兩種模式

5-5	修復 Emma 提出的 Bug	⭐⭐⭐

依據 **Emma** 的 **5-E1, 5-E2 bug report** 修復計算邏輯錯誤

- 每個 bug 先寫失敗的測試 (Red)，再修復 (Green)

5-6	Integration Tests	⭐⭐⭐⭐

E2E 測試流程：

```
// 記帳流程 E2E:
testWidgets('full bookkeeping flow', (tester) async {
  // 1. 開啟 App
  // 2. 點擊「+」按鈕
  // 3. 輸入金額 150
  // 4. 選擇分類「飲食」
  // 5. 點擊「儲存」
  // 6. 驗證總覽頁餘額更新
  // 7. 驗證報表頁圓餅圖有飲食分類
});
```

5-7	CI/CD Pipeline	⭐⭐⭐⭐

```
# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  lint-and-test:
    steps:
      - dart analyze
      - dart format --set-exit-if-changed
      - flutter test --coverage
      - 檢查覆蓋率 ≥ 80%
  build:
    needs: lint-and-test
    steps:
      - flutter build ios --release
      - flutter build appbundle --release
  deploy: (on tag only)
    - fastlane upload to TestFlight / Play Console
```

5-8	App Icon 設計	⭐⭐

設計可愛的倉鼠保險箱 App Icon，用 flutter_launcher_icons 自動產生各尺寸。

✅ 全域 Checklist

架構與安全

- Isar AES-256 加密，key 存 Keychain/Keystore
- API Key 用 --dart-define 注入
- SSL Pinning 對 Yahoo Finance / frankfurter.app
- App 背景時清除敏感 UI
- Jailbreak/Root 偵測警告

效能

- 股價查詢 debounce 300ms
- 交易列表 infinite scroll, 每次 30 筆
- Isar Index: Transaction.dateTime, categoryId, accountId
- Riverpod select / family 縮小 rebuild 範圍

可觀測性

- Sentry crash reporting (privacy-first)
- logger: Debug 詳細、Release 只記 error
- Dio Interceptor 記錄 request/response 時間
- 日誌中絕不記錄財務數據

無障礙

- Semantics label 對所有資產數字
- WCAG 2.1 AA 對比度 (≥ 4.5:1)
- 支援系統字型縮放
- 可點擊區域最小 48x48dp

💪 Let's build something amazing, Erin! 🐹